Python

Quick Reference for Data Analysis & Statistics

numpy · pandas · matplotlib · seaborn · plotnine

Computational Biology Course · All indexing is 0-based

0-based indexing: The first element of any list, array or DataFrame row is always at index `[0]`, not [1]. Slices are *exclusive* at the end: `v[1:4]` returns elements 1, 2, 3 (not 4).

1. General Commands & Built-ins

Python	Description / Notes
`x = 5`	Assign a value to a variable.
`x, y = 3, 7` `a = b = 0`	Multiple assignment and chained assignment.
`print(x)` `print(x, y, sep=', ')`	Print to console. Use sep= to customise the separator between values.
`type(x)`	Return the type/class of an object: int, float, str, list, etc.
`isinstance(x, int)`	Check if x is an instance of a given type. Returns True or False.
`len(x)`	Number of elements in a string, list, tuple, dict, or array.
`del x`	Remove a variable from the namespace.
`dir()`	List all variables / names in the current namespace.
`help(func)` `func.__doc__`	Display documentation for a function or object.
`# This is a comment`	Single-line comment. Python has no multi-line comment syntax — use multiple #.

`import math`
`from math import sqrt`
`import numpy as np`
	Import a module, import a specific function, or import with an alias.

`import os`
`os.getcwd()`
`os.chdir('path')`
	Get or change the current working directory.

`os.makedirs('folder',` `exist_ok=True)`	Create a directory (including parent directories).
`import os` `os.listdir('.')`	List files and folders in a directory.

`str(x)`
`int(x)`
`float(x)`
`bool(x)`
	Type conversion / casting between basic types.

`f'{name} is {age} years old'` `f'{value:.2f}'`	f-strings: modern string formatting. :.2f formats to 2 decimal places.
`'hello' + ' ' + 'world'`	String concatenation with + operator.

`'hello'.upper()`
`'HELLO'.lower()`
`'hello world'.title()`
	Common string methods: uppercase, lowercase, title case.

Python	Description / Notes

`'  hello  '.strip()`
`'a,b,c'.split(',')`
`','.join(['a','b','c'])`
	Strip whitespace, split on a delimiter, join a list into a string.

`'hello'.replace('l', 'r')`
`'hello'.startswith('he')`
`'lo' in 'hello'`
	String replace, starts-with test, and membership test.

`open('f.txt', 'r').read()` `open('f.txt', 'w').write('hi')`	Read or write a text file. Prefer 'with open(...)' for safety.
`with open('file.txt', 'r') as f:` `  content = f.read()`	Recommended file-reading pattern — file is closed automatically.
`import sys` `sys.argv`	Access command-line arguments passed to the script.

2. Mathematical Functions

Import math for standard maths functions. For array maths use numpy (Section 3).

Python	Description / Notes
`import math`	Import the standard math module before using functions below.
`math.sqrt(x)`	Square root. Alternatively: x ** 0.5
`x ** 2` `x ** 0.5`	Power operator. x**2 = x^2, x**0.5 = $\sqrt{x}$.
`abs(x)`	Absolute value. Works on int, float, and complex numbers.
`math.exp(x)`	Exponential: e^x.

`math.log(x)`
`math.log(x, base)`
`math.log10(x)`
`math.log2(x)`
	Natural log, log with arbitrary base, log base-10, log base-2.

`math.ceil(x)`
`math.floor(x)`
`round(x, n)`
	Ceiling, floor, and rounding to n decimal places.

`x % y`	Modulo — remainder after division. E.g. 10 % 3 == 1
`x // y`	Integer (floor) division. E.g. 10 // 3 == 3
`math.factorial(n)`	Factorial of n: n!
`math.comb(n, k)`	Binomial coefficient: n choose k. Available from Python 3.8+.

`max(a, b)`
`min(a, b)`
`max(iterable)`
	Maximum and minimum. Works on two values or any iterable.

`sum(iterable)`	Sum of all elements in a list or other iterable.

`math.pi`
`math.e`
`math.tau`
	Mathematical constants: π, e, and τ (= 2π).

`float('inf')`
`float('-inf')`
`float('nan')`
	Special float values: positive infinity, negative infinity, not-a-number.

Python	Description / Notes
`math.isnan(x)` `math.isinf(x)`	Test for NaN or infinity.

`math.sin(x)`
`math.cos(x)`
`math.tan(x)`
	Trigonometric functions. Argument is in radians.

`math.degrees(x)` `math.radians(x)`	Convert between radians and degrees.

`import random`
`random.random()`
`random.randint(1, 10)`
`random.choice(lst)`
	Random number generation: uniform [0,1), random integer, random element.

`random.seed(42)`	Set the random seed for reproducibility.

3. Arrays & Vectors — numpy

import numpy as np · numpy applies arithmetic element-wise by default (vectorised operations).

Python	Description / Notes
`import numpy as np`	Import numpy — required before all np.* calls below.
`np.array([1, 2, 3])`	Create a 1-D array from a Python list.
`np.array([[1,2],[3,4]])`	Create a 2-D array (matrix) from a nested list.

`np.zeros(5)`
`np.ones((3, 4))`
`np.full((2, 2), 7)`
	Arrays of zeros, ones, or a constant value. Pass a tuple for 2-D shape.

`np.arange(1, 11)` `np.arange(0, 1, 0.1)`	Integer or float sequence. End value is exclusive (like Python range).
`np.linspace(0, 1, 50)`	50 evenly spaced values between 0 and 1 inclusive.

`np.random.rand(5)`
`np.random.randn(5)`
`np.random.randint(0,10,5)`
	Random arrays: uniform [0,1), standard normal, random integers.

`np.random.seed(42)`	Set random seed for reproducibility.

`v.shape`
`v.ndim`
`v.dtype`
`len(v)`
	Array shape (tuple), number of dimensions, data type, length of first axis.

`v[0]`
`v[-1]`
`v[1:4]`
`v[::2]`
	Indexing (0-based), last element, slicing, every 2nd element.

`m[1, 2]`
`m[0, :]`
`m[:, 1]`
	2-D indexing: element at row 1 col 2, entire row 0, entire col 1.

`v[v > 3]` `v[(v > 1) & (v < 5)]`	Boolean / conditional indexing. Combines conditions with & (and) | (or).

`v * 2`
`v + 10`
`v ** 2`
`np.sqrt(v)`
	Vectorised arithmetic — operations apply element-wise automatically.

Python	Description / Notes
`v1 + v2` `v1 * v2`	Element-wise addition and multiplication of two arrays.
`np.dot(v1, v2)` `m1 @ m2`	Dot product of two vectors; matrix multiplication with @ operator.

`np.sum(v)`
`np.mean(v)`
`np.std(v, ddof=1)`
`np.var(v, ddof=1)`
	Aggregations. ddof=1 gives the sample (unbiased) standard deviation / variance.

`np.median(v)`
`np.min(v)`
`np.max(v)`
	Median, minimum, maximum.

`np.quantile(v, 0.25)`	Quantile / percentile. 0.25 = first quartile (Q1).
`np.sort(v)` `np.argsort(v)`	Sorted array; indices that would sort the array.
`np.unique(v)` `np.unique(v, return_counts=True)`	Unique values; unique values with their occurrence counts.
`np.where(v > 3)`	Indices of elements satisfying a condition.
`np.concatenate([a, b])` `np.stack([a, b])`	Concatenate arrays end-to-end; stack as new rows.
`v.reshape(3, 4)` `v.flatten()`	Reshape to new dimensions (total elements must match); flatten to 1-D.
`np.isnan(v)` `np.nan_to_num(v)`	Detect NaN values; replace NaN with 0 (or a given value).
`np.append(v, x)`	Return new array with x appended. Does NOT modify v in place.
`np.repeat(v, 3)` `np.tile(v, 3)`	Repeat each element 3 times; repeat the whole array 3 times.
`np.cumsum(v)` `np.cumprod(v)`	Cumulative sum; cumulative product.
`np.corrcoef(x, y)` `np.cov(x, y)`	Pearson correlation matrix; covariance matrix.

`np.linalg.inv(m)`
`np.linalg.det(m)`
`np.linalg.eig(m)`
	Matrix inverse, determinant, and eigenvalues/eigenvectors.

4. Data Frames — pandas

import pandas as pd · DataFrame = table with labelled rows and columns. Series = single labelled column.

Python	Description / Notes
`import pandas as pd`	Import pandas — required before all pd.* calls below.
`pd.DataFrame({'a':[1,2],'b':['x','y']})`	Create a DataFrame from a dictionary of column-name → list pairs.
`pd.read_csv('file.csv')` `pd.read_csv('f.csv', sep=';', header=0)`	Read a CSV file. sep= sets delimiter; header= sets which row is the header.
`df.to_csv('file.csv', index=False)`	Write DataFrame to CSV. index=False omits the row-number column.

Python	Description / Notes
`pd.read_excel('file.xlsx')` `df.to_excel('file.xlsx', index=False)`	Read / write Excel files. Requires openpyxl package.

`df.shape`
`len(df)`
`len(df.columns)`
	Dimensions: (rows, cols) tuple; number of rows; number of columns.

`df.columns`
`df.index`
`df.dtypes`
	Column names; row index; data type of each column.

`df.head()` `df.tail(10)`	First 5 rows (default); last 10 rows.
`df.info()` `df.describe()`	Column types and non-null counts; statistical summary of numeric columns.
`df['col']` `df[['col1','col2']]`	Select one column (returns Series); select multiple columns (returns DataFrame).

`df.iloc[0]`
`df.iloc[0:5]`
`df.iloc[2, 3]`
	Row(s) by integer position; rows 0–4; element at row 2, col 3.

`df.loc[0]` `df.loc[0:5, 'col']`	Row(s) by label/index; rows 0–5 of a specific column (label-based, inclusive).
`df[df['age'] > 30]` `df.query('age > 30')`	Filter rows by condition; .query() uses a string expression (often cleaner).
`df.loc[df['age']>30, ['name','age']]`	Filter rows AND select specific columns simultaneously.
`df['new'] = df['a'] * 2` `df = df.assign(new=df['a']*2)`	Add or overwrite a column. .assign() returns a new DataFrame (non-mutating).
`df.drop('col', axis=1)` `df.drop(0, axis=0)`	Drop a column (axis=1) or a row by index label (axis=0).
`df.rename(columns={'old':'new'})`	Rename one or more columns.
`df.sort_values('col')` `df.sort_values('col', ascending=False)`	Sort by a column, ascending (default) or descending.
`pd.merge(df1, df2, on='id')` `pd.merge(df1, df2, on='id', how='left')`	Inner join (default) or left join on a key column.
`pd.concat([df1, df2])` `pd.concat([df1, df2], axis=1)`	Stack DataFrames vertically (append rows) or horizontally (append columns).
`df.groupby('grp')['y'].mean()` `df.groupby('grp').agg({'y':'mean','x':'sum'})`	Group by a column and aggregate. .agg() allows per-column functions.
`df.groupby('g').agg(m=('y','mean'),` `n=('y','count'))`	Named aggregations — control output column names explicitly.
`df.pivot_table(values='v', index='r',` `columns='c', aggfunc='mean')`	Pivot table: reshape long data into a wide summary table.
`df.melt(id_vars=['id'],` `value_vars=['a','b'])`	Melt wide format to long format (opposite of pivot).

`df.isna()`
`df.isna().sum()`
`df.dropna()`
`df.fillna(0)`
	Detect NaN; count missing per column; drop rows with NaN; fill NaN with 0.

`df.duplicated()` `df.drop_duplicates()`	Boolean mask of duplicate rows; remove duplicate rows.

Python	Description / Notes
`df['col'].value_counts()`	Frequency count of each unique value in a column.
`df['col'].astype(float)` `df['col'].astype('category')`	Cast a column to a different type. 'category' saves memory for low-cardinality columns.
`pd.to_datetime(df['date'])` `df['date'].dt.year`	Convert a column to datetime type; extract year (also .month, .day, .hour…).
`df.apply(func, axis=1)` `df['col'].map(func)`	.apply() applies a function row-wise or column-wise; .map() applies element-wise to a Series.

5. Plotting — matplotlib · seaborn · plotnine

Three levels: matplotlib (full control), seaborn (DataFrame-aware, high-level), plotnine (ggplot2 grammar).

Python	Description / Notes

`import matplotlib.pyplot as plt`
`import seaborn as sns`
`from plotnine import *`
	Import the three main plotting libraries. plotnine mirrors R's ggplot2 grammar.

`# ── matplotlib ──────────────────`	Low-level, full control. Foundation for seaborn.
`plt.plot(x, y)` `plt.plot(x, y, 'r--', linewidth=2)`	Line plot. Format string: colour + linestyle. 'r--' = red dashed.
`plt.scatter(x, y)` `plt.scatter(x, y, c=colours, s=sizes, alpha=0.7)`	Scatter plot. c= maps colours, s= maps point sizes, alpha= sets transparency.
`plt.hist(x, bins=20, edgecolor='white')`	Histogram. bins= sets number of bins; edgecolor adds white bar borders.
`plt.bar(categories, values)` `plt.barh(categories, values)`	Vertical and horizontal bar charts.
`plt.boxplot(data)` `plt.violinplot(data)`	Box plot and violin plot from a list of arrays.

`plt.title('Title')`
`plt.xlabel('X axis')`
`plt.ylabel('Y axis')`
	Set plot title and axis labels.

`plt.xlim(0, 100)` `plt.ylim(0, 50)`	Set axis limits manually.
`plt.xscale('log')` `plt.yscale('log')`	Logarithmic axis scale.
`plt.legend()` `plt.legend(['A','B'], loc='upper right')`	Show legend. Use label= in plot calls, or pass label list explicitly.

`plt.tight_layout()`
`plt.savefig('fig.png', dpi=300)`
`plt.show()`
	Fix layout overlap; save to file at high resolution; display the plot.

`fig, ax = plt.subplots(1, 2, figsize=(10,4))`
`ax[0].plot(x, y)`
`ax[1].scatter(x, y)`
	Object-oriented API: create a figure with 2 subplots side by side.

`plt.style.use('ggplot')` `plt.style.use('seaborn-v0_8-whitegrid')`	Apply a built-in visual style to approximate ggplot2 or seaborn aesthetics.

Python	Description / Notes
`# ── seaborn ──────────────────────`	High-level, DataFrame-aware. Closest to ggplot2 in style.
`sns.set_theme(style='whitegrid')` `sns.set_palette('husl')`	Set global theme and colour palette.
`sns.scatterplot(x='a', y='b',` `hue='group', data=df)`	Scatter plot. hue= maps a column to colour automatically.
`sns.lineplot(x='x', y='y',` `hue='group', data=df)`	Line plot with optional grouping by colour.
`sns.histplot(x='weight', data=df,` `binwidth=5, hue='diet')`	Histogram. binwidth= controls bin size; hue= splits by group colour.
`sns.kdeplot(x='weight', data=df,` `hue='diet', fill=True)`	Kernel density estimate (smooth histogram). fill=True shades under the curve.
`sns.boxplot(x='diet', y='weight',` `data=df)`	Box plot grouped by a categorical variable.

`sns.violinplot(x='diet',`
`y='weight',`
`data=df)`
	Violin plot — shows full distribution shape per group.

`sns.regplot(x='age', y='weight',` `data=df, ci=95)`	Scatter plot with a regression line and confidence band (ci= sets level).

`sns.heatmap(corr_matrix,`
`annot=True,`
`cmap='coolwarm')`
	Heatmap — great for correlation matrices.

`g = sns.FacetGrid(df, col='diet')` `g.map(sns.histplot, 'weight')`	Faceted plots — one panel per group, equivalent to ggplot2 facet_wrap.
`sns.pairplot(df, hue='group')`	Grid of scatter plots for every pair of numeric columns.
`# ── plotnine (ggplot2 in Python) ─`	pip install plotnine — mirrors R's ggplot2 grammar almost exactly.

`ggplot(df, aes(x='age',`
`y='weight'))`
`  + geom_point(alpha=0.7)`
	Scatter plot. Note: column names must be quoted strings in Python.

`ggplot(df, aes(x='weight',`
`fill='diet'))`
`  + geom_histogram(binwidth=5,`
`color='white')`
	Histogram coloured by diet group.

`... + geom_smooth(method='lm',` `se=True)`	Add a linear regression line with confidence band.
`... + facet_wrap('~ diet')`	Split into separate panels by diet group.
`... + labs(title='T', x='X', y='Y')` `  + theme_bw()`	Add labels and apply a clean black-and-white theme.

6. Flow Control, Functions & Exceptions

Blocks are defined by indentation (4 spaces), not curly braces. Forgetting the colon : after if/for/def is the most common beginner syntax error.

Python	Description / Notes
`if x > 0:` `  print('positive')`	Basic if statement. Colon + indented block — no curly braces needed.

Python	Description / Notes

`if x > 0:`
`  print('positive')`
`else:`
`  print('non-positive')`
	if / else. The else block runs when the condition is False.

`if x > 0:`
`  print('positive')`
`elif x == 0:`
`  print('zero')`
`else:`
`  print('negative')`
	elif (else-if) — Python's way to chain multiple conditions. No limit on elif branches.

`result = 'pos' if x > 0 else 'neg'`	Ternary / inline conditional: value_if_true if condition else value_if_false.

`match day:`
`  case 'Mon': print('Monday')`
`  case 'Tue': print('Tuesday')`
`  case _:  print('Other')`
	match / case (Python 3.10+). case _ is the default catch-all, like 'else'.

`match point:`
`  case (0, 0):  print('origin')`
`  case (x, 0):  print(f'x={x}')`
`  case (x, y):  print(f'{x},{y}')`
	match can destructure tuples and capture variables — much more powerful than a simple switch.

`for i in range(10):` `  print(i)`	for loop over range(n) produces 0, 1, … n-1.
`for i in range(1, 11):` `  print(i)`	range(start, stop) — produces 1 to 10 (stop is exclusive).
`for i in range(0, 20, 2):` `  print(i)`	range(start, stop, step) — even numbers 0, 2, 4 … 18.
`for item in ['a', 'b', 'c']:` `  print(item)`	Iterate directly over any iterable (list, tuple, string, array…).
`for i, val in enumerate(lst):` `  print(i, val)`	enumerate() gives index and value together — avoids manual counter variables.
`for a, b in zip(lst1, lst2):` `  print(a, b)`	zip() pairs up elements from two (or more) iterables simultaneously.
`while x > 0:` `  x -= 1`	while loop — repeats as long as condition is True.

`while True:`
`  do_something()`
`  if not cond:`
`  break`
	Infinite loop with break — Python's equivalent of do/while (runs at least once).

`break`	Exit the current loop immediately.
`continue`	Skip the rest of the current iteration and jump to the next one.
`for i in range(5):` `  pass`	pass is a no-op placeholder — useful for empty blocks that will be filled in later.
`[x**2 for x in range(10)]`	List comprehension — concise alternative to a for loop that builds a list.
`[x for x in lst if x > 0]`	Filtered list comprehension — only include elements matching the condition.
`{k: v for k, v in d.items()}`	Dictionary comprehension — build a dict from an iterable.
`{x**2 for x in range(10)}`	Set comprehension — like list comprehension but produces a unique set.

Python	Description / Notes
`def greet(name, greeting='Hello'):` `  return f'{greeting}, {name}!'`	Function with a default argument. Python always needs an explicit return statement.

`def stats(lst):`
`  return min(lst), max(lst), sum(lst)/len(lst)`
 	
`lo, hi, avg = stats(data)`
	Return multiple values as a tuple; unpack them with multiple assignment.

`lambda x: x ** 2` `sorted(lst, key=lambda x: x[1])`	Anonymous function (lambda). Most useful as a short key function for sorting etc.

`*args and **kwargs:`
`def f(*args, **kwargs):`
`  print(args, kwargs)`
	*args collects extra positional arguments as a tuple; **kwargs collects keyword arguments as a dict.

`try:`
`  result = 10 / x`
`except ZeroDivisionError:`
`  print('divide by zero!')`
	Catch a specific exception type. Always prefer catching specific exceptions over bare except.

`try:`
`  risky()`
`except ValueError as e:`
`  print(e)`
`except Exception as e:`
`  print('other:', e)`
`finally:`
`  cleanup()`
	Multiple except clauses catch different error types. finally always runs regardless of outcome.

`raise ValueError('bad input')`	Raise an exception manually to signal an error condition.

7. Statistics — scipy.stats · statsmodels · sklearn

pip install scipy statsmodels scikit-learn · scipy.stats for tests & distributions, statsmodels for regression tables, sklearn for ML pipelines.

Python	Description / Notes
`# ── Descriptive Statistics ────`	scipy.stats and numpy cover most needs. Install: pip install scipy
`import numpy as np` `from scipy import stats`	Core imports for statistics. scipy.stats has distributions, tests, and more.

`np.mean(x)`
`np.median(x)`
`np.std(x, ddof=1)`
`np.var(x, ddof=1)`
	Mean, median, sample std dev, sample variance. ddof=1 applies Bessel's correction (divides by n-1).

`stats.mode(x)`	Mode — most frequent value. Returns ModeResult with .mode and .count attributes.
`np.percentile(x, [25, 50, 75])` `np.quantile(x, 0.95)`	Quartiles and arbitrary quantiles. percentile takes 0-100 scale; quantile takes 0-1 scale.
`stats.describe(x)`	All-in-one summary: n, min, max, mean, variance, skewness, kurtosis.
`stats.skew(x)` `stats.kurtosis(x)`	Skewness (symmetry) and excess kurtosis (tail heaviness) of a sample.
`stats.pearsonr(x, y)`	Pearson correlation coefficient. Returns (r, p-value). Assumes linearity and normality.
`stats.spearmanr(x, y)` `stats.kendalltau(x, y)`	Spearman rank correlation and Kendall tau — non-parametric alternatives to Pearson.

Python	Description / Notes
`np.cov(x, y)[0, 1]`	Covariance between x and y. np.cov returns the full 2x2 matrix; [0,1] extracts the off-diagonal.
`# ── Probability Distributions ──`	Each distribution object exposes: .pdf/.pmf, .cdf, .ppf (quantile), .rvs (random samples).

`stats.norm(loc=0, scale=1)`
`stats.norm.pdf(x)`
`stats.norm.cdf(1.96)`
`stats.norm.ppf(0.975)`
	Normal distribution. loc=mean, scale=std. pdf=density, cdf=area to left, ppf=inverse cdf.

`stats.norm.rvs(loc=0, scale=1,` `  size=1000)`	Draw 1000 random samples from a normal distribution.
`stats.t(df=10)` `stats.t.ppf(0.975, df=10)`	Student's t-distribution. ppf gives the critical value for a two-tailed 5% significance test.
`stats.chi2(df=5)` `stats.chi2.ppf(0.95, df=5)`	Chi-squared distribution. ppf gives the critical value at the 95th percentile.
`stats.f(dfn=2, dfd=20)`	F-distribution. dfn = numerator degrees of freedom, dfd = denominator.

`stats.binom(n=10, p=0.5)`
`stats.binom.pmf(k=3, n=10, p=0.5)`
`stats.binom.cdf(k=3, n=10, p=0.5)`
	Binomial distribution. pmf gives P(X=3); cdf gives P(X<=3).

`stats.poisson(mu=4.5)`
`stats.expon(scale=2.0)`
`stats.uniform(loc=0, scale=1)`
	Poisson, Exponential (scale=1/lambda), and Uniform distributions.

`# ── t-tests ──────────────────────`	Tests on means. All return (statistic, p-value). Reject H0 if p-value < alpha (typically 0.05).
`stats.ttest_1samp(x, popmean=0)`	One-sample t-test: is the mean of x significantly different from popmean?

`stats.ttest_ind(x, y)`
`stats.ttest_ind(x, y,`
`  equal_var=False)`
	Independent two-sample t-test (equal variances).
	Welch's t-test (unequal variances — safer default choice).

`stats.ttest_rel(x, y)`	Paired t-test: same subjects measured twice (before/after). x and y must have equal length.

`t_stat, p_val = stats.ttest_ind(x, y)`
`if p_val < 0.05:`
`  print('Reject H0')`
`else:`
`  print('Fail to reject H0')`
	Interpret the result: compare p-value to your significance level alpha.

`# ── Normality Tests ──────────────`	Test whether data are consistent with a normal distribution (H0 = data is normal).
`stats.shapiro(x)`	Shapiro-Wilk test. Most powerful for small samples (n < 50). Returns (W-statistic, p-value).
`stats.normaltest(x)`	D'Agostino-Pearson test combining skewness and kurtosis. Better for larger samples.
`stats.kstest(x, 'norm',` `  args=(mu, sigma))`	Kolmogorov-Smirnov test against any named distribution. args= passes distribution parameters.
`# ── Non-parametric Tests ────────`	Use when normality is violated, sample is small, or data is ordinal.
`stats.mannwhitneyu(x, y,` `  alternative='two-sided')`	Mann-Whitney U test — non-parametric equivalent of the independent samples t-test.
`stats.wilcoxon(x, y)`	Wilcoxon signed-rank test — non-parametric equivalent of the paired t-test.

Python	Description / Notes
`stats.kruskal(g1, g2, g3)`	Kruskal-Wallis H test — non-parametric equivalent of one-way ANOVA.
`stats.friedmanchisquare(g1, g2, g3)`	Friedman test — non-parametric equivalent of repeated-measures ANOVA.
`# ── Chi-squared Tests ──────────`	Tests for categorical data.

`stats.chisquare(`
`  f_obs=[20, 30, 50],`
`  f_exp=[25, 25, 50])`
	Goodness-of-fit chi-squared: are observed frequencies consistent with expected frequencies?

`from scipy.stats import`
`chi2_contingency`
`chi2, p, dof, expected =`
`  chi2_contingency(table)`
	Test of independence. table is a 2-D array (contingency table). Returns chi2, p, df, expected counts.

`# ── ANOVA ──────────────────────`	Analysis of Variance — test whether 3+ group means are equal. H0: all means are equal.
`stats.f_oneway(g1, g2, g3)`	One-way ANOVA using scipy. Returns (F-statistic, p-value). Simple but gives no ANOVA table.

`import statsmodels.api as sm`
`from statsmodels.formula.api`
`  import ols`
 	
`model = ols('weight ~ C(diet)',`
`  data=df).fit()`
`sm.stats.anova_lm(model, typ=1)`
	One-way ANOVA via statsmodels. Returns a full ANOVA table with SS, df, MS, F, and p-value.

`model = ols(`
`  'weight ~ C(diet) + C(sex)'`
`  ' + C(diet):C(sex)',`
`  data=df).fit()`
`sm.stats.anova_lm(model, typ=2)`
	Two-way ANOVA with interaction term. C() wraps categorical variables. typ=2 = Type II SS.

`from statsmodels.stats.anova`
`  import AnovaRM`
 	
`AnovaRM(df, 'score', 'subject',`
`  within=['time']).fit()`
	Repeated-measures ANOVA. Requires long-format DataFrame with a subject ID column.

`from statsmodels.stats.multicomp`
`  import pairwise_tukeyhsd`
 	
`res = pairwise_tukeyhsd(`
`  df['weight'], df['diet'])`
`print(res.summary())`
	Tukey HSD post-hoc test — run after significant ANOVA to identify which group pairs differ.

`from statsmodels.stats.multicomp`
`  import multipletests`
 	
`multipletests(p_values,`
`  method='fdr_bh')`
	Multiple comparison correction. 'fdr_bh' = Benjamini-Hochberg FDR. Also: 'bonferroni', 'holm'.

`# ── Linear Regression ──────────`	pip install statsmodels scikit-learn for the functions below.

`from scipy.stats import linregress`
 	
`slope, intercept, r, p, se =`
`  linregress(x, y)`
	Simple linear regression with scipy. Returns slope, intercept, r, p-value, and standard error of slope.

`import statsmodels.api as sm`
 	
`X = sm.add_constant(x)`
`model = sm.OLS(y, X).fit()`
`print(model.summary())`
	OLS regression with statsmodels. add_constant() adds the intercept column. summary() gives full table.

Python	Description / Notes

`from statsmodels.formula.api import ols`
 	
`model = ols('weight ~ age + C(sex)', data=df).fit()`
`print(model.summary())`
	Multiple regression with R-style formula. C() marks categorical predictors. Intercept included by default.

`model.params`
`model.pvalues`
`model.rsquared`
`model.rsquared_adj`
`model.conf_int(alpha=0.05)`
	Extract coefficients, p-values, R-squared, adjusted R-squared, and 95% confidence intervals.

`model.resid`
`model.fittedvalues`
`model.predict(new_df)`
	Residuals, fitted values, and predictions on new data. Use residuals for diagnostic plots.

`# ── sklearn Regression ──────────`	sklearn prioritises prediction pipelines over inference tables.

`from sklearn.linear_model import LinearRegression`
 	
`model = LinearRegression()`
`model.fit(X, y)`
`model.score(X, y)`
`model.coef_`
`model.intercept_`
	Fit, evaluate (R^2), and inspect coefficients. X must be 2-D: reshape with X.reshape(-1,1) for 1 predictor.

`from sklearn.linear_model import Ridge, Lasso`
 	
`Ridge(alpha=1.0).fit(X, y)`
`Lasso(alpha=0.1).fit(X, y)`
	Regularised regression. Ridge (L2) shrinks coefficients; Lasso (L1) sets some to zero (feature selection).

`from sklearn.model_selection import train_test_split`
 	
`X_tr, X_te, y_tr, y_te = train_test_split(`
`X, y, test_size=0.2,`
`random_state=42)`
	Split into training (80%) and test (20%) sets. random_state ensures reproducibility.

`from sklearn.metrics import (mean_squared_error, r2_score)`
 	
`rmse = mean_squared_error(y_te, y_pred) ** 0.5`
`r2 = r2_score(y_te, y_pred)`
	Evaluate regression: RMSE (in same units as y) and R^2 (1 = perfect, 0 = baseline mean model).

`# ── Logistic Regression ─────────`	For binary outcomes (0/1). Models the probability P(y=1 | X).

`from sklearn.linear_model import LogisticRegression`
 	
`model = LogisticRegression()`
`model.fit(X_train, y_train)`
`y_pred = model.predict(X_test)`
`y_prob = model.predict_proba(X_test)[:, 1]`
	Fit, predict class labels, and predict class probabilities (column 1 = P(class=1)).

`from sklearn.metrics import (accuracy_score, confusion_matrix, classification_report)`
 	
`accuracy_score(y_test, y_pred)`
`confusion_matrix(y_test, y_pred)`
	Classification metrics: accuracy, confusion matrix, and full report with precision, recall, F1-score.

Python	Description / Notes
`print(classification_report(` `  y_test, y_pred))`	

`import statsmodels.api as sm`
 	
`result = sm.Logit(`
`  y, sm.add_constant(X)`
`).fit()`
`print(result.summary())`
	Logistic regression via statsmodels — provides coefficient table with z-statistics, p-values, and CIs.